

lecture_17.py



```

1  import os
2  import sys
3  from typing import Callable
4  import math
5  from dataclasses import dataclass
6  import torch
7  import torch.nn as nn
8  from torch.nn import functional as F
9  from torch.nn.functional import softmax
10 from einops import einsum, rearrange, repeat
11 from execute_util import text, link, image
12 from lecture_util import named_link
13 from references import ppo2017, grpo, qwen3, llama3
14 import matplotlib.pyplot as plt
15 from tqdm import tqdm
16
17 def main():
18     Last lecture: overview of RL from verifiable rewards (policy gradient)
19     This lecture: deep dive into the mechanics of policy gradient (e.g., GRPO)
20
21     rl_setup_for_language_models()
22     policy_gradient()
23     training_walkthrough()
24
25     Summary
26     • Reinforcement learning is the key to surpassing human abilities
27     • If you can measure it, you can optimize it
28     • Policy gradient framework is conceptually clear, just need baselines to reduce variance
29     • RL systems is much more complex than pretraining (inference workloads, manage multiple models)
30
31     Final two lectures:
32     • Junyang Lin (Qwen) [Yang+ 2025]
33     • Mike Lewis (Llama) [Grattafiori+ 2024]
34
35
36 def rl_setup_for_language_models():
37     State s: prompt + generated response so far
38     Action a: generate next token
39
40     Rewards R: how good the response is; we'll focus on:
41     • Outcome rewards, which depend on the entire response
42     • Verifiable rewards, whose computation is deterministic
43     • Notions of discounting and bootstrapping are less applicable
44     Example: "... Therefore, the answer is 3 miles."
45
46     Transition probabilities  $T(s' | s, a)$ : deterministic  $s' = s + a$ 
47     • Can do planning / test-time compute (unlike in robotics)
48     • States are really made up (different from robotics), so a lot of flexibility
49
50     Policy  $\pi(a | s)$ : just a language model (fine-tuned)
51
52     Rollout/episode/trajectory:  $s \rightarrow a \rightarrow \dots \rightarrow a \rightarrow a \rightarrow R$ 
53     Objective: maximize expected reward  $E[R]$ 
54     (where the expectation is taken over prompts  $s$  and response tokens  $a$ )
55
56
57 def policy_gradient():
58     For notational simplicity, let  $a$  denote the entire response.
59
60     We want to maximize expected reward with respect to the policy  $\pi$ :

```

$$E[R] = \int p(s) \pi(a | s) R(s, a)$$

Obvious thing to do is to take the gradient:

$$\nabla E[R] = \int p(s) \nabla \pi(a | s) R(s, a)$$

$$\nabla E[R] = \int p(s) \pi(a | s) \nabla \log \pi(a | s) R(s, a)$$

$$\nabla E[R] = E[\nabla \log \pi(a | s) R(s, a)]$$

Naive policy gradient:

- Sample prompt s , sample response $a \sim \pi(a | s)$
- Update parameters based on $\nabla \log \pi(a | s) R(s, a)$ (same as SFT, but weighted by $R(s, a)$)

Setting: $R(s, a) \in \{0, 1\}$ = whether response is correct or not

- Naive policy gradient only updates on correct responses
- Like SFT, but dataset changing over time as policy changes

Challenge: high noise/variance

In this setting, sparse rewards (few responses get reward 1, most get 0)

In contrast: in RLHF, reward models (learned from pairwise preferences) are more continuous

Baselines

Recall $\nabla E[R] = E[\nabla \log \pi(a | s) R(s, a)]$

$\nabla \log \pi(a | s) R(s, a)$ is an unbiased estimate of $\nabla E[R]$, but maybe there are others with lower variance...

Example: two states

- s_1 : $a_1 \rightarrow$ reward 11, $a_2 \rightarrow$ reward 9
- s_2 : $a_1 \rightarrow$ reward 0, $a_2 \rightarrow$ reward 2

Don't want $s_1 \rightarrow a_2$ (reward 9) because a_1 is better, want $s_2 \rightarrow a_2$ (reward 2), but $9 > 2$

Idea: maximize the baselined reward: $E[R - b(s)]$

This is just $E[R]$ shifted by a constant $E[b(s)]$ that doesn't depend on the policy π

We update based on $\nabla \log \pi(a | s) (R(s, a) - b(s))$

What $b(s)$ should we use?

Example: two states

Assuming uniform distribution over (s, a) and $|\nabla \pi(a | s)| = 1$

```
naive_variance = torch.std(torch.tensor([11., 9, 0, 2])) # @inspect naive_variance
```

Define baseline $b(s_1) = 10$, $b(s_2) = 1$

```
baseline_variance = torch.std(torch.tensor([11. - 10, 9 - 10, 0 - 1, 2 - 1])) # @inspect baseline_variance
```

Variance reduced from 5.323 to 1.155

Optimal $b^*(s) = E[(\nabla \pi(a | s))^2 R | s] / E[(\nabla \pi(a | s))^2 | s]$ (for one-parameter models)

This is difficult to compute...

...so heuristic is to use the mean reward:

$$b(s) = E[R | s]$$

This is still hard to compute and must be estimated.

Advantage functions

This choice of $b(s)$ has connections to advantage functions.

- $V(s) = E[R | s]$ = expected reward from state s
 - $Q(s, a) = E[R | s, a]$ = expected reward from state s taking action a
- (Note: Q and R are the same here, because we're assuming a has all actions and we have outcome rewards.)

Definition (advantage): $A(s, a) = Q(s, a) - V(s)$

Intuition: how much better is action a than expected from state s

If $b(s) = E[R | s]$, then the baselined reward is identical to the advantage!

$$E[R - b(s)] = A(s, a)$$

In general:

- Ideal: $E[\nabla \log \pi(a | s) R(s, a)]$

- Estimate: $\nabla \log \pi(a | s) \delta$
- There are multiple choices of δ , as we'll see later.

[\[CS224R lecture notes\]](#)

```
def training_walkthrough():
```

Group Relative Policy Optimization (GRPO) [\[Shao+ 2024\]](#)

- Simplification to PPO that removes the critic (value function)
- Leverages the group structure in the LM setting (multiple responses per prompt), which provides a natural baseline $b(s)$.

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_{ϕ} ; task prompts $\mathcal{D}$; hyperparameters ϵ, β, μ

```
1: policy model  $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$ 
2: for iteration = 1, ..., I do
3:   reference model  $\pi_{\text{ref}} \leftarrow \pi_{\theta}$ 
4:   for step = 1, ..., M do
5:     Sample a batch  $\mathcal{D}_b$  from  $\mathcal{D}$ 
6:     Update the old policy model  $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ 
7:     Sample  $G$  outputs  $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$  for each question  $q \in \mathcal{D}_b$ 
8:     Compute rewards  $\{r_i\}_{i=1}^G$  for each sampled output  $o_i$  by running  $r_{\phi}$ 
9:     Compute  $\hat{A}_{i,t}$  for the  $t$ -th token of  $o_i$  through group relative advantage estimation.
10:    for GRPO iteration = 1, ...,  $\mu$  do
11:      Update the policy model  $\pi_{\theta}$  by maximizing the GRPO objective (Equation 21)
12:    Update  $r_{\phi}$  through continuous training using a replay mechanism.
```

Output π_{θ}

```
simple_task()      # Define a simple task
simple_model()     # Define a simple model
```

Let's now define the GRPO algorithm.

```
run_policy_gradient(num_epochs=1, num_steps_per_epoch=1)
```

Let's actually train some models.

```
experiments()
```

```
def simple_task():
```

Task: sorting n numbers

Prompt: n numbers

```
prompt = [1, 0, 2]
```

Response: n numbers

```
response = [0, 1, 2]
```

Reward should capture how close to sorted the response is.

Define a reward that returns the number of positions where the response matches the ground truth.

```
reward = sort_distance_reward([3, 1, 0, 2], [0, 1, 2, 3]) # @inspect reward
```

```
reward = sort_distance_reward([3, 1, 0, 2], [7, 2, 2, 5]) # @inspect reward @stepover
```

```
reward = sort_distance_reward([3, 1, 0, 2], [0, 3, 1, 2]) # @inspect reward @stepover
```

Define an alternative reward that gives more partial credit.

```
reward = sort_inclusion_ordering_reward([3, 1, 0, 2], [0, 1, 2, 3]) # @inspect reward
```

```
reward = sort_inclusion_ordering_reward([3, 1, 0, 2], [7, 2, 2, 5]) # @inspect reward @stepover
```

```
reward = sort_inclusion_ordering_reward([3, 1, 0, 2], [0, 3, 1, 2]) # @inspect reward @stepover
```

Note that the second reward function provides more credit to the 3rd response than the first reward function.

```
def simple_model():
```

Define a simple model that maps prompts to responses

- Assume fixed prompt and response length
- Captures positional information with separate per-position parameters
- Decode each position in the response independently (not autoregressive)

```

173     model = Model(vocab_size=3, embedding_dim=10, prompt_length=3, response_length=3)
174
175     Start with a prompt s
176     prompts = torch.tensor([[1, 0, 2]]) # [batch pos]
177
178     Generate responses a
179     torch.manual_seed(10)
180     responses = generate_responses(prompts=prompts, model=model, num_responses=5) # [batch trial pos] @inspect responses
181
182     Compute rewards R of these responses:
183     rewards = compute_reward(prompts=prompts, responses=responses, reward_fn=sort_inclusion_ordering_reward) # [batch trial]
@inspect rewards
184
185     Compute deltas  $\delta$  given the rewards R (for performing the updates)
186     deltas = compute_deltas(rewards=rewards, mode="rewards") # [batch trial] @inspect deltas
187     deltas = compute_deltas(rewards=rewards, mode="centered_rewards") # [batch trial] @inspect deltas
188     deltas = compute_deltas(rewards=rewards, mode="normalized_rewards") # [batch trial] @inspect deltas
189     deltas = compute_deltas(rewards=rewards, mode="max_rewards") # [batch trial] @inspect deltas
190
191     Compute log probabilities of these responses:
192     log_probs = compute_log_probs(prompts=prompts, responses=responses, model=model) # [batch trial] @inspect log_probs
193
194     Compute loss so that we can use to update the model parameters
195     loss = compute_loss(log_probs=log_probs, deltas=deltas, mode="naive") # @inspect loss
196
197     freezing_parameters()
198
199     old_model = Model(vocab_size=3, embedding_dim=10, prompt_length=3, response_length=3) # Pretend this is an old checkpoint
@stepover
200     old_log_probs = compute_log_probs(prompts=prompts, responses=responses, model=old_model) # @stepover
201     loss = compute_loss(log_probs=log_probs, deltas=deltas, mode="unclipped", old_log_probs=old_log_probs) # @inspect loss
202     loss = compute_loss(log_probs=log_probs, deltas=deltas, mode="clipped", old_log_probs=old_log_probs) # @inspect loss
203
204     Sometimes, we can use an explicit KL penalty to regularize the model.
205     This can be useful if you want RL a new capability into a model, but you don't want it to forget its original
    capabilities.
206      $KL(p \parallel q) = E_{\{x \sim p\}}[\log(p(x)/q(x))]$ 
207      $KL(p \parallel q) = E_{\{x \sim p\}}[-\log(q(x)/p(x))]$ 
208      $KL(p \parallel q) = E_{\{x \sim p\}}[\log(p(x)/q(x)) - \log(q(x)/p(x)) - 1]$  because  $E_{\{x \sim p\}}[\log(q(x)/p(x))] = 1$ 
209     kl_penalty = compute_kl_penalty(log_probs=log_probs, ref_log_probs=old_log_probs) # @inspect kl_penalty
210
211     Summary:
212     • Generate responses
213     • Compute rewards R and  $\delta$  (rewards, centered rewards, normalized rewards, max rewards)
214     • Compute log probs of responses
215     • Compute loss from log probs and  $\delta$  (naive, unclipped, clipped)
216
217
218     def freezing_parameters():
219         Motivation: in GRPO you'll see ratios:  $p(a | s) / p_{old}(a | s)$ 
220         When you're optimizing, it is important to freeze and not differentiate through p_old
221         w = torch.tensor(2., requires_grad=True)
222         p = torch.nn.Sigmoid()(w)
223         p_old = torch.nn.Sigmoid()(w)
224         ratio = p / p_old
225         ratio.backward()
226         grad = w.grad # @inspect grad
227
228     Do it properly:
229     w = torch.tensor(2., requires_grad=True)
230     p = torch.nn.Sigmoid()(w)
231     with torch.no_grad(): # Important: treat p_old as a constant!
232         p_old = torch.nn.Sigmoid()(w)
233     ratio = p / p_old

```

```

234     ratio.backward()
235     grad = w.grad # @inspect grad
236
237
238 def compute_reward(prompts: torch.Tensor, responses: torch.Tensor, reward_fn: Callable[[list[int], list[int]], float]) ->
torch.Tensor:
239     """
240     Args:
241         prompts (int[batch pos])
242         responses (int[batch trial pos])
243     Returns:
244         rewards (float[batch trial])
245     """
246     batch_size, num_responses, _ = responses.shape
247     rewards = torch.empty(batch_size, num_responses, dtype=torch.float32)
248     for i in range(batch_size):
249         for j in range(num_responses):
250             rewards[i, j] = reward_fn(prompts[i, :], responses[i, j, :])
251     return rewards
252
253
254 def sort_distance_reward(prompt: list[int], response: list[int]) -> float: # @inspect prompt, @inspect response
255     """
256     Return how close response is to ground_truth = sorted(prompt).
257     In particular, compute number of positions where the response matches the ground truth.
258     """
259     assert len(prompt) == len(response)
260     ground_truth = sorted(prompt)
261     return sum(1 for x, y in zip(response, ground_truth) if x == y)
262
263
264 def sort_inclusion_ordering_reward(prompt: list[int], response: list[int]) -> float: # @inspect prompt, @inspect response
265     """
266     Return how close response is to ground_truth = sorted(prompt).
267     """
268     assert len(prompt) == len(response)
269
270     # Give one point for each token in the prompt that shows up in the response
271     inclusion_reward = sum(1 for x in prompt if x in response) # @inspect inclusion_reward
272
273     # Give one point for each adjacent pair in response that's sorted
274     ordering_reward = sum(1 for x, y in zip(response, response[1:]) if x <= y) # @inspect ordering_reward
275
276     return inclusion_reward + ordering_reward
277
278
279 class Model(nn.Module):
280     def __init__(self, vocab_size: int, embedding_dim: int, prompt_length: int, response_length: int):
281         super().__init__()
282         self.embedding_dim = embedding_dim
283         self.embedding = nn.Embedding(vocab_size, embedding_dim)
284         # For each position, we have a matrix for encoding and a matrix for decoding
285         self.encode_weights = nn.Parameter(torch.randn(prompt_length, embedding_dim, embedding_dim) / math.sqrt(embedding_dim))
286         self.decode_weights = nn.Parameter(torch.randn(response_length, embedding_dim, embedding_dim) / math.sqrt(embedding_dim))
287
288     def forward(self, prompts: torch.Tensor) -> torch.Tensor:
289         """
290         Args:
291             prompts: int[batch pos]
292         Returns:
293             logits: float[batch pos vocab]
294         """
295         # Embed the prompts
296         embeddings = self.embedding(prompts) # [batch pos dim]

```

```

297
298     # Transform using per prompt position matrix, collapse into one vector
299     encoded = einsum(embeddings, self.encode_weights, "batch pos dim1, pos dim1 dim2 -> batch dim2")
300
301     # Turn into one vector per response position
302     decoded = einsum(encoded, self.decode_weights, "batch dim2, pos dim2 dim1 -> batch pos dim1")
303
304     # Convert to logits (input and output share embeddings)
305     logits = einsum(decoded, self.embedding.weight, "batch pos dim1, vocab dim1 -> batch pos vocab")
306
307     return logits
308
309
310 def generate_responses(prompts: torch.Tensor, model: Model, num_responses: int) -> torch.Tensor:
311     """
312     Args:
313         prompts (int[batch pos])
314     Returns:
315         generated responses: int[batch trial pos]
316
317     Example (batch_size = 3, prompt_length = 3, num_responses = 2, response_length = 4)
318     p1 p1 p1 r1 r1 r1 r1
319         r2 r2 r2 r2
320     p2 p2 p2 r3 r3 r3 r3
321         r4 r4 r4 r4
322     p3 p3 p3 r5 r5 r5 r5
323         r6 r6 r6 r6
324     """
325     logits = model(prompts) # [batch pos vocab]
326     batch_size = prompts.shape[0]
327
328     # Sample num_responses (independently) for each [batch pos]
329     flattened_logits = rearrange(logits, "batch pos vocab -> (batch pos) vocab")
330     flattened_responses = torch.multinomial(torch.softmax(flattened_logits, dim=-1), num_samples=num_responses, replacement=True) #
[batch pos trial]
331     responses = rearrange(flattened_responses, "(batch pos) trial -> batch trial pos", batch=batch_size)
332     return responses
333
334
335 def compute_log_probs(prompts: torch.Tensor, responses: torch.Tensor, model: Model) -> torch.Tensor:
336     """
337     Args:
338         prompts (int[batch pos])
339         responses (int[batch trial pos])
340     Returns:
341         log_probs (float[batch trial pos]) under the model
342     """
343     # Compute log prob of responses under model
344     logits = model(prompts) # [batch pos vocab]
345     log_probs = F.log_softmax(logits, dim=-1) # [batch pos vocab]
346
347     # Replicate to align with responses
348     num_responses = responses.shape[1]
349     log_probs = repeat(log_probs, "batch pos vocab -> batch trial pos vocab", trial=num_responses) # [batch trial pos vocab]
350
351     # Index into log_probs using responses
352     log_probs = log_probs.gather(dim=-1, index=responses.unsqueeze(-1)).squeeze(-1) # [batch trial pos]
353
354     return log_probs
355
356
357 def compute_deltas(rewards: torch.Tensor, mode: str) -> torch.Tensor: # @inspect rewards
358     """
359     Args:

```

```

360         rewards (float[batch trial])
361     Returns:
362         deltas (float[batch trial]) which are advantage-like quantities for updating
363     """
364     if mode == "rewards":
365         return rewards
366
367     if mode == "centered_rewards":
368         # Compute mean over all the responses (trial) for each prompt (batch)
369         mean_rewards = rewards.mean(dim=-1, keepdim=True) # @inspect mean_rewards
370         centered_rewards = rewards - mean_rewards # @inspect centered_rewards
371         return centered_rewards
372
373     if mode == "normalized_rewards":
374         mean_rewards = rewards.mean(dim=-1, keepdim=True) # @inspect mean_rewards
375         std_rewards = rewards.std(dim=-1, keepdim=True) # @inspect std_rewards
376         centered_rewards = rewards - mean_rewards # @inspect centered_rewards
377         normalized_rewards = centered_rewards / (std_rewards + 1e-5) # @inspect normalized_rewards
378         return normalized_rewards
379
380     if mode == "max_rewards":
381         # Zero out any reward that isn't the maximum for each batch
382         max_rewards = rewards.max(dim=-1, keepdim=True)[0]
383         max_rewards = torch.where(rewards == max_rewards, rewards, torch.zeros_like(rewards))
384         return max_rewards
385
386     raise ValueError(f"Unknown mode: {mode}")
387
388
389 def compute_loss(log_probs: torch.Tensor, deltas: torch.Tensor, mode: str, old_log_probs: torch.Tensor | None = None) ->
torch.Tensor:
390     if mode == "naive":
391         return -einsum(log_probs, deltas, "batch trial pos, batch trial -> batch trial pos").mean()
392
393     if mode == "unclipped":
394         ratios = torch.exp(log_probs - old_log_probs) # [batch trial]
395         return -einsum(ratios, deltas, "batch trial pos, batch trial -> batch trial pos").mean()
396
397     if mode == "clipped":
398         epsilon = 0.01
399         unclipped_ratios = torch.exp(log_probs - old_log_probs) # [batch trial]
400         unclipped = einsum(unclipped_ratios, deltas, "batch trial pos, batch trial -> batch trial pos")
401
402         clipped_ratios = torch.clamp(unclipped_ratios, min=1 - epsilon, max=1 + epsilon)
403         clipped = einsum(clipped_ratios, deltas, "batch trial pos, batch trial -> batch trial pos")
404         return -torch.minimum(unclipped, clipped).mean()
405
406     raise ValueError(f"Unknown mode: {mode}")
407
408 def compute_kl_penalty(log_probs: torch.Tensor, ref_log_probs: torch.Tensor) -> torch.Tensor:
409     """
410     Compute an estimate of KL(model | ref_model), where the models are given by:
411         log_probs [batch trial pos vocab]
412         ref_log_probs [batch trial pos vocab]
413     Use the estimate:
414         KL(p || q) = E_p[q/p - log(q/p) - 1]
415     """
416     return (torch.exp(ref_log_probs - log_probs) - (ref_log_probs - log_probs) - 1).sum(dim=-1).mean()
417
418
419 def run_policy_gradient(num_epochs: int = 100,
420                        num_steps_per_epoch: int = 10,
421                        compute_ref_model_period: int = 10,
422                        num_responses: int = 10,

```

```

423         deltas_mode: str = "rewards",
424         loss_mode: str = "naive",
425         kl_penalty: float = 0.0,
426         reward_fn: Callable[[list[int], list[int]], float] = sort_inclusion_ordering_reward,
427         use_cache: bool = False) -> tuple[str, str]:
428     """Train a model using policy gradient.
429     Return:
430     - Path to the image of the learning curve.
431     - Path to the log file
432     """
433     torch.manual_seed(5)
434
435     image_path = f"var/policy_gradient_{deltas_mode}_{loss_mode}.png"
436     log_path = f"var/policy_gradient_{deltas_mode}_{loss_mode}.txt"
437
438     # Already ran, just cache it
439     if use_cache and os.path.exists(image_path) and os.path.exists(log_path):
440         return image_path, log_path
441
442     # Define the data
443     prompts = torch.tensor([[1, 0, 2], [3, 2, 4], [1, 2, 3]])
444     vocab_size = prompts.max() + 1
445     prompt_length = response_length = prompts.shape[1]
446
447     model = Model(vocab_size=vocab_size, embedding_dim=10, prompt_length=prompt_length, response_length=response_length)
448     optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
449
450     records = []
451     ref_log_probs = None
452     ref_model = None
453     old_log_probs = None
454
455     if use_cache:
456         out = open(log_path, "w")
457     else:
458         out = sys.stdout
459
460     for epoch in tqdm(range(num_epochs), desc="epoch"):
461         # If using KL penalty, need to get the reference model (freeze it every few epochs)
462         if kl_penalty != 0:
463             if epoch % compute_ref_model_period == 0:
464                 ref_model = model.clone()
465
466         # Sample responses and evaluate their rewards
467         responses = generate_responses(prompts=prompts, model=model, num_responses=num_responses) # [batch trial pos]
468         rewards = compute_reward(prompts=prompts, responses=responses, reward_fn=reward_fn) # [batch trial]
469         deltas = compute_deltas(rewards=rewards, mode=deltas_mode) # [batch trial]
470
471         if kl_penalty != 0: # Compute under the reference model
472             with torch.no_grad():
473                 ref_log_probs = compute_log_probs(prompts=prompts, responses=responses, model=ref_model) # [batch trial]
474
475         if loss_mode != "naive": # Compute under the current model (but freeze while we do the inner steps)
476             with torch.no_grad():
477                 old_log_probs = compute_log_probs(prompts=prompts, responses=responses, model=model) # [batch trial]
478
479         # Take a number of steps given the responses
480         for step in range(num_steps_per_epoch):
481             log_probs = compute_log_probs(prompts=prompts, responses=responses, model=model) # [batch trial]
482             loss = compute_loss(log_probs=log_probs, deltas=deltas, mode=loss_mode, old_log_probs=old_log_probs) # @inspect loss
483             if kl_penalty != 0:
484                 loss += kl_penalty * compute_kl_penalty(log_probs=log_probs, ref_log_probs=ref_log_probs)
485
486         # Print information

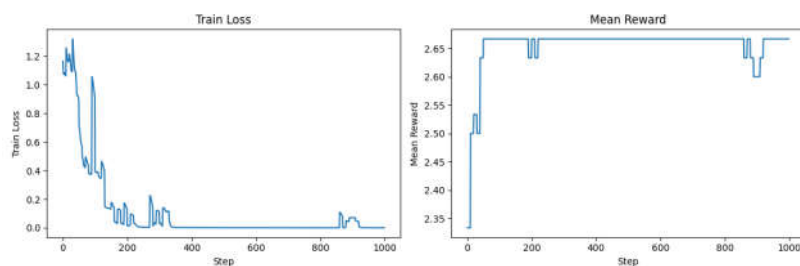
```

```

487         print_information(epoch=epoch, step=step, loss=loss, prompts=prompts, rewards=rewards, responses=responses,
log_probs=log_probs, deltas=deltas, out=out)
488         global_step = epoch * num_steps_per_epoch + step
489         records.append({"epoch": epoch, "step": global_step, "loss": loss.item(), "mean_reward": rewards.mean().item()})
490
491         # Backprop and update parameters
492         optimizer.zero_grad()
493         loss.backward()
494         optimizer.step()
495     if use_cache:
496         out.close()
497
498     if use_cache:
499         # Plot step versus loss and reward in two subplots
500         steps = [r["step"] for r in records]
501         losses = [r["loss"] for r in records]
502         rewards = [r["mean_reward"] for r in records]
503
504         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
505
506         # Loss subplot
507         ax1.plot(steps, losses)
508         ax1.set_xlabel("Step")
509         ax1.set_ylabel("Train Loss")
510         ax1.set_title("Train Loss")
511
512         # Reward subplot
513         ax2.plot(steps, rewards)
514         ax2.set_xlabel("Step")
515         ax2.set_ylabel("Mean Reward")
516         ax2.set_title("Mean Reward")
517
518         plt.tight_layout()
519         plt.savefig(image_path)
520         plt.close()
521
522     return image_path, log_path
523
524
525 def print_information(epoch: int, step: int, loss: torch.Tensor, prompts: torch.Tensor, rewards: torch.Tensor, responses:
torch.Tensor, log_probs: torch.Tensor, deltas: torch.Tensor, out):
526     print(f"epoch = {epoch}, step = {step}, loss = {loss:.3f}, reward = {rewards.mean():.3f}", file=out)
527     if epoch % 1 == 0 and step % 5 == 0:
528         for batch in range(prompts.shape[0]):
529             print(f"    prompt = {prompts[batch, :]}", file=out)
530             for trial in range(responses.shape[1]):
531                 print(f"        response = {responses[batch, trial, :]}, log_probs = {tstr(log_probs[batch, trial])}, reward =
{rewards[batch, trial]}, delta = {deltas[batch, trial]:.3f}", file=out)
532
533
534 def tstr(x: torch.Tensor) -> str:
535     return "[" + ", ".join(f"{x[i]:.3f}" for i in range(x.shape[0])) + "]"
536
537
538 def experiments():
539     Let's start with updating based on raw rewards.
540     image_path, log_path = run_policy_gradient(num_epochs=100, num_steps_per_epoch=10, num_responses=10, deltas_mode="rewards",
loss_mode="naive", reward_fn=sort_inclusion_ordering_reward, use_cache=True) # @stepover

```

541



[var/policy_gradient_rewards_naive.txt](#)

542

Looking through the output, you'll see that by the end, we haven't really learned sorting very well (and this is still the training set).

543

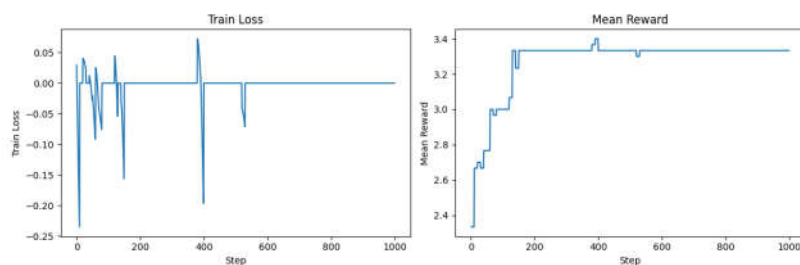
544

Let's try using centered rewards.

545

```
image_path, log_path = run_policy_gradient(num_epochs=100, num_steps_per_epoch=10, num_responses=10,
deltas_mode="centered_rewards", loss_mode="naive", reward_fn=sort_inclusion_ordering_reward, use_cache=True) # @stepover
```

546



[var/policy_gradient_centered_rewards_naive.txt](#)

547

This seems to help, as:

548

- Suboptimal rewards get a negative gradient update, and

549

- If all the responses for a given prompt have the same reward, then we don't update.

550

Overall, this is better, but we're still getting stuck in local optima.

551

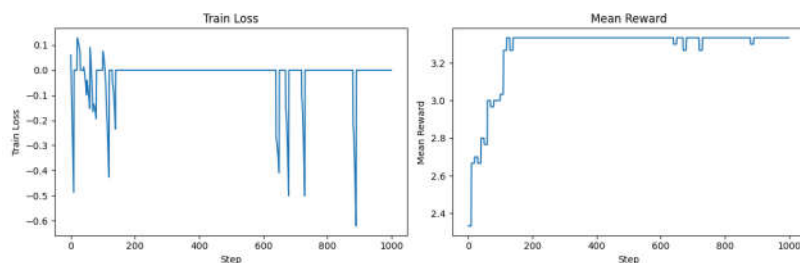
552

Finally, let's try normalizing by the standard deviation.

553

```
image_path, log_path = run_policy_gradient(num_epochs=100, num_steps_per_epoch=10, num_responses=10,
deltas_mode="normalized_rewards", loss_mode="naive", reward_fn=sort_inclusion_ordering_reward, use_cache=True) # @stepover
```

554



[var/policy_gradient_normalized_rewards_naive.txt](#)

555

There is not much difference here, and indeed, variants like Dr. GRPO do not perform this normalization to avoid length bias (not an issue here since all responses have the same length. [\[Liu+ 2025\]](#))

556

557

Overall, as you can see, reinforcement learning is not trivial, and you can easily get stuck in suboptimal states.

558

The hyperparameters could probably be tuned better...

559

560

```
561 if __name__ == "__main__":
```

562

```
    main()
```